

Structural Anomalies in a Windows 10/11 System Binary Corpus: Cross-Build Outliers and a `inpoutx64.sys` Case Study

Abdelrahman Khalid 
abdnul108@gmail.com

September 2026

Abstract

Windows system binaries form a large and evolving population whose deviations from structural and security convention are difficult to observe at corpus scale. This paper presents a corpus-scale study of structural anomalies within the shared 15,695-binary Windows 10/11 corpus, examining binaries and binary clusters whose properties depart from corpus-wide convention, including an extended case study of `inpoutx64.sys`, a kernel-mode driver whose anomalies extend into questions of provenance, capability, and trust infrastructure. The results demonstrate how corpus-wide structural baselines can surface anomalies, and their security implications, that are difficult to identify through manual, single-binary inspection alone.

1 Introduction

Paper one of this series established a corpus-scale baseline for the structural and security characteristics of 15,695 Windows system binaries, showing that most binaries conform to well-defined structural conventions and that security-relevant configuration trends toward stricter defaults between Windows 10 and Windows 11 [1]. That baseline describes the corpus in aggregate. It does not, by construction, say anything about the binaries that depart from it, or about what such departures reveal when examined individually rather than folded into a population-wide statistic. This paper is concerned with that remainder.

Windows’ scale is itself part of what makes the remainder worth examining. The operating system distributes tens of thousands of binaries, authored across a mix of internal teams and external vendors, many carried forward largely unmodified across multiple releases, and maintained without the kind of ecosystem-wide, centrally reviewable visibility that an open codebase would provide. Structural and security consistency across a catalog of this size is not a property that can be assumed; it is a property that has to survive the accumulated decisions of every team, vendor, and legacy component that has ever touched the shipped image, indefinitely and without a single point of review. A binary shipping unsigned, a data directory that fails to parse cleanly, or a driver whose shipped capability set diverges from its own author’s account of what he wrote, are treated in this paper as observable symptoms of that difficulty rather than as unrelated curiosities.

These anomalies also bear on a narrower question: what a passing trust signal actually certifies. A valid Authenticode chain, a conventional-looking structure, or a recognized signer are ordinarily treated, implicitly, as evidence that a binary is safe to trust. Each is in fact a narrower claim than that. A signature establishes that a binary has not been altered since a specific point in time; it says nothing about whether that binary’s behavior was ever reviewed by anyone beyond its author, or whether it will continue to be treated as trustworthy indefinitely. The anomalies examined here – particularly the extended case study in § 3.3 – make this gap concrete: a kernel-mode driver can remain Authenticode-valid, absent from the current vulnerable-driver blocklist, and unflagged by Windows own built-in antimalware component, while exposing a capability set its shipped binary’s own author disclaims having written.

The remainder of this paper proceeds as follows. § 2 describes the shared corpus and the three distinct routes by which the anomalies below were surfaced. § 3 presents the anomalies themselves: binaries observed unsigned in the Windows 10 build, most but not all absent from Windows 11, a set of isolated field-level contradictions, and an extended case study of `inpoutx64.sys`. § 4 and § 5 discuss the limitations of this approach and situate it against related work, and § 6 concludes.

2 Methodology

2.1 Corpus and Analysis Pipeline

This paper draws on the same 15,695-binary corpus and extraction pipeline described in [1], collected from Windows 10 (build 10.0.19045) and Windows 11 (build 10.0.26200) live installations and released via the accompanying repository [2]. Corpus scope, collection method, and associated limitations are not repeated here; readers evaluating the anomalies below against the base corpus should consult § 2 of [1].

2.2 Anomaly Detection

The anomalies examined in this paper were surfaced through three distinct routes rather than a single uniform detection method, and each is described separately below so that a reader can judge what kind of claim each anomaly type supports.

Unsigned Binaries (§ 3.1). Signing status for every binary in the corpus was determined through a two-stage verification procedure rather than a single check for an embedded signature. Each binary was first checked for an embedded Authenticode signature via `WinVerifyTrust` (`WTD_STATEACTION_VERIFY`, with revocation checking disabled via `WTD_REVOKE_NONE` for batch throughput across the full corpus). Binaries that failed this check specifically with `TRUST_E_NOSIGNATURE` – meaning no embedded signature was present, as opposed to an invalid or untrusted one – were then checked against the system catalog store: the file’s SHA-256 hash was computed and looked up via `CryptCATAdminEnumCatalogFromHash`, and if a matching catalog entry was found, the binary was re-verified against that `.cat` file directly to obtain a fresh trust result. Only binaries that failed *both* the embedded check and the catalog fallback were classified as unsigned. This two-stage procedure is what rules out catalog-signature false negatives: Windows system binaries are frequently signed via catalog rather than embedded signature, and a single-stage embedded-only check would misclassify such binaries as unsigned. The 14 binaries in § 3.1 are exclusively those that failed both stages in the Windows 10 build; nine are absent from the Windows 11 corpus entirely, while the remaining five fail both stages in Windows 11 as well.

Field-Level Anomalies (§ 3.2). The three binaries examined in § 3.2 were not found through a targeted anomaly scan. They surfaced incidentally during corpus construction, as edge cases that the parsing pipeline did not initially handle correctly – structurally valid PE files whose field values violated implicit assumptions the parser had made about well-formed input. Each such case required a parser fix to handle correctly; rather than discarding these three binaries once the parser was corrected, they were retained as anomaly candidates because the underlying field values – not the parser’s initial failure to handle them – were themselves unconventional. This detection route is opportunistic rather than exhaustive: it surfaces whatever the parser’s assumptions happen to be insufficiently permissive for, not a defined statistical or rule-based criterion applied uniformly across the corpus. This asymmetry is discussed as a limitation in § 4.

inputx64.sys (§ 3.3). Unlike the other two routes, `inputx64.sys` was surfaced through a targeted, corpus-wide automated check for RWX section permissions, cross-referenced against signer recognition and driver functional category. The full discovery process and the reasoning behind each filtering step are given in § 3.3 (*Discovery*) rather than repeated here, since that walkthrough is itself part of the case study’s findings.

2.3 Case Study Methodology: `inputx64.sys`

`inputx64.sys` was selected for extended treatment because its anomalies extend beyond structural irregularity into questions of provenance and capability. To characterize it, the shipped, signed binary was manually reverse-engineered directly via Ghidra: its dispatch routine, physical-memory mapping logic, and stack-frame geometry were reconstructed from decompiled output and raw disassembly (Appendix A), without reference to any existing source release, allowing inference of the compiler flags and security features used in the original build. The binary’s PE header compile timestamp was cross-referenced against Highresolution Enterprises’ documented release history for the signed v1.2.0.0 build [3]. Separately, the original author was contacted directly and provided the source for an earlier, related project (`hwinterface`), predating the `inputx64.sys` rename; this source was used only as a comparison baseline to identify

functionality present in the shipped binary but absent from it (§ 3.3, *Capability Delta Against Author-Provided Source*), not as a basis for the reconstruction itself. The binary’s inclusion in the LOLDrivers BYOVD database [4] was used as external corroboration of its security relevance, not as a basis for the structural findings themselves.

3 Structural Anomalies

With the corpus, detection routes, and `inputx64.sys` reconstruction methodology established in § 2, this section presents the anomalies themselves. The section proceeds as follows:

- § 3.1 examines binaries observed unsigned in the Windows 10 corpus, most but not all absent from Windows 11;
- § 3.2 examines individual binaries with malformed or unconventional structural fields that nonetheless parse as valid;
- § 3.3 presents an extended case study of `inputx64.sys`, whose anomalous characteristics merit deeper investigation;

3.1 Unsigned Binaries Across the Windows 10/11 Corpus

Windows binaries are generally expected to carry a valid digital signature, whether via embedded Authenticode or catalog-based signing [5]. Beyond the signed population characterized in § 3.5 Signature Statistics, Table 1 shows that 14 binaries were flagged as unsigned in the Windows 10 build (signing status determined per the verification procedure in § 2, ruling out catalog-signature false negatives). All 14 warrant individual discussion for their kernel-mode or hardware-facing role. Cross-referencing this set against the Windows 11 build shows two distinct outcomes: `FreeImage.dll`, both `br_bandlux` drivers, `ftser2k.sys`, `GdmFilt.sys`, `gdmfwup.sys`, `massfilter.sys`, `usbaapl64.sys`, and `usbscan.sys` (nine of fourteen) are absent entirely from the Windows 11 corpus, consistent with removal or replacement of the underlying component. The remaining five — `D2D1Debug2.dll`, `d3d11sdklayers.dll`, `d3d11_1sdklayers.dll`, `d3d11_2sdklayers.dll`, and `bthmodem.sys` — persist as unsigned in the Windows 11 build as well, indicating an unresolved rather than a transient condition for this subset.

#1 – Windows/System32/FreeImage.dll	
Type	DLL
Ring	User-mode
Likely role	Open-source image codec library (FreeImage project)
Signing risk	Low – unsigned but no kernel access; ordinary user-mode DLL-hijack/supply-chain territory
#2 – Windows/SysWOW64/D2D1Debug2.dll	
Type	DLL
Ring	User-mode
Likely role	Direct2D debug/validation layer
Signing risk	Low – debug-only component, not typically loaded in production paths
#3 – Windows/SysWOW64/d3d11sdklayers.dll	
Type	DLL
Ring	User-mode
Likely role	Direct3D 11 SDK debug layer
Signing risk	Low – dev/debug tooling
#4 – Windows/SysWOW64/d3d11_1sdklayers.dll	
Type	DLL
Ring	User-mode
Likely role	D3D11.1 SDK debug layer
Signing risk	Low – same category as above
#5 – Windows/SysWOW64/d3d11_2sdklayers.dll	

Continued on next page

Table 1 – continued from previous page

Type	DLL
Ring	User-mode
Likely role	D3D11.2 SDK debug layer
Signing risk	Low – same category as above
#6 – Windows/System32/drivers/br_bandlux_cpo.sys	
Type	Driver
Ring	Kernel (Ring 0)
Likely role	Bandlux (Compal) mobile broadband/WWAN modem driver – vendor-specific cellular modem support
Signing risk	High – OEM/vendor driver, hardware-facing, wouldn't load under default DSE while unsigned
#7 – Windows/System32/drivers/br_bandlux_dc_enum.sys	
Type	Driver
Ring	Kernel (Ring 0)
Likely role	Companion enumerator for the Bandlux modem stack (device/composite enumeration)
Signing risk	High – same vendor family as #6, likely paired bus/enumeration driver
#8 – Windows/System32/drivers/bthmodem.sys	
Type	Driver
Ring	Kernel (Ring 0)
Likely role	Bluetooth modem driver (Microsoft-authored; models a Bluetooth device as a modem/COM port)
Signing risk	High – Microsoft-origin but unsigned here is notable; touches Bluetooth hardware stack
#9 – Windows/System32/drivers/ftser2k.sys	
Type	Driver
Ring	Kernel (Ring 0)
Likely role	FTDI USB-to-serial (UART) converter driver – common third-party USB serial chip driver
Signing risk	High – classic USB-facing vendor driver; FTDI drivers have real-world history of driver-level issues/counterfeiting controversies
#10 – Windows/System32/drivers/GdmFilt.sys	
Type	Driver
Ring	Kernel (Ring 0)
Likely role	Modem filter driver (commonly associated with mobile broadband/GSM modem stacks, e.g. Sierra Wireless/Gobi-family)
Signing risk	High – filter drivers sit in the device stack and can intercept/modify I/O to underlying hardware
#11 – Windows/System32/drivers/gdmfwup.sys	
Type	Driver
Ring	Kernel (Ring 0)
Likely role	Companion "firmware update" driver for the same modem family as #10
Signing risk	High – firmware-update-capable drivers imply a path to modifying attached hardware's firmware
#12 – Windows/System32/drivers/massfilter.sys	
Type	Driver
Ring	Kernel (Ring 0)
Likely role	USB mass-storage filter driver (sits in the USB mass storage device stack)
Signing risk	High – directly relevant to the USB-scanning angle; filter drivers here can observe/modify mass-storage I/O
#13 – Windows/System32/drivers/usbaapl64.sys	

Continued on next page

Table 1 – continued from previous page

Type	Driver
Ring	Kernel (Ring 0)
Likely role	Apple Mobile Device USB driver (enables Windows to recognize iPhone/iPad over USB)
Signing risk	High – widely deployed (installed by iTunes/Apple Mobile Device Support), directly USB-facing, high real-world install base despite unsigned in this corpus instance
#14 – Windows/System32/drivers/usbscan.sys	
Type	Driver
Ring	Kernel (Ring 0)
Likely role	Microsoft-authored USB scanner class driver (WIA scanner support)
Signing risk	High – Microsoft-origin, directly matches the ”scans USB hardware” angle; notable that even a Microsoft class driver appears unsigned here

Table 1: Binaries observed unsigned in the Windows 10 corpus.

It is worth noting that the presence of these binaries in the corpus does not necessarily indicate a systemic Windows characteristic. Since the corpus was collected from a live, in-use operating system rather than a clean reference image, some of these entries could reflect third-party software having installed or modified the binary post-deployment, rather than the binary shipping unsigned by default. False positives arising from catalog signature lookup limitations cannot be fully ruled out either. This caveat applies most directly to the nine binaries absent from Windows 11, whose removal could equally reflect deliberate deprecation or an artifact of this particular machine’s software history; the five binaries persisting as unsigned across both builds are less readily explained this way, since the same unsigned condition would need to have survived unmodified across a full OS upgrade. As such, this finding should be read as an observation specific to the sampled environment rather than a general claim about Windows as shipped.

3.2 Field-Level Anomalies

Three binaries in the corpus exhibit isolated field-level contradictions — structurally valid PE metadata in most respects, punctuated by a single data directory or descriptor entry that is either malformed or self-contradictory. Each case required a targeted relaxation of the parsing pipeline’s logic to process past it without aborting, and each is unique within the 15,695-binary corpus. The following subsections examine each in turn.

3.2.1 MaxxAudioEQ64.dll — Malformed Relocation Directory

Identification.

Field	Value
Filename	MaxxAudioEQ64.dll
Path	C:\Windows\System32\MaxxAudioEQ64.dll
Windows Build	10.0.19045
SHA-256	6970180b6e2222cefe0cafcf143b15d3ab1b8d7e23f5a1df7e4431e7d46ac2f5
MD5	56eae185cece57aeedef15bf1f772c9d
PE Timestamp	0x53465DC4 (2014-04-10 09:00:52 UTC)

Table 2: Identification of the anomalous binary.

Observation The `IMAGE_DIRECTORY_ENTRY_BASERELOC` data directory entry reports the following raw values, extracted directly from the Optional Header:

Field	Raw Value	Decimal
Directory VirtualAddress	0x9FEF9C37	2,683,280,439
Directory Size	0xC212EDFF	3,256,020,479
Resolved Entry Count	0	—
File Offset (attempted)	0xCB000	831,488

Table 3: Raw `IMAGE_BASE_RELOCATION` directory values as parsed by the analysis pipeline.

For context, the binary’s computed in-memory image size is 2,068,480 bytes. The claimed directory `VirtualAddress` therefore exceeds the entire mapped image by a factor of approximately $1,297\times$, and the claimed `Size` field exceeds it by approximately $1,574\times$ — both values are unconditionally out of bounds for any valid in-memory reference. This is the only binary in the corpus for which the analysis pipeline’s bounds-checking logic required an explicit relaxation to permit continued processing rather than aborting on this directory.

Structural Context

Metric	Value
Disk File Size	2,051,704 bytes
In-Memory Size	2,068,480 bytes
Section Count	9 (3 executable, 1 writable)
File Alignment	0x200
Section Alignment	0x1000
Overlay Present	Yes (13,944 bytes at offset 0x1F1800)

Table 4: Structural summary of `MaxxAudioEQ64.dll`, for reference.

Discussion The remainder of the binary’s structure is unremarkable: alignment, section count, and overlay presence all fall within normal ranges observed elsewhere in the corpus. The anomaly is therefore isolated entirely to this single data directory entry rather than reflecting broader structural corruption.

Notably, the binary’s PE timestamp (0x53465DC4, corresponding to 2014-04-10) predates both Windows builds examined in this corpus by over a decade. This suggests the binary has shipped essentially unmodified across multiple Windows releases, which offers a plausible — though not conclusively provable — explanation for why a malformed directory of this severity could persist undetected: the binary likely predates whatever build or link toolchain conventions produce well-formed relocation directories elsewhere in the corpus, and has simply never been recompiled or re-linked since.

Whereas `MaxxAudioEQ64.dll`’s anomaly manifests as an out-of-bounds data directory entry, the following binary exhibits a narrower, single-descriptor contradiction within an otherwise well-formed import table.

3.2.2 `ir41_32original.dll` — Import Descriptor with Valid Thunks but Zero Entries

Identification.

Field	Value
Filename	<code>ir41_32original.dll</code>
Path	<code>C:\Windows\SysWOW64\ir41_32original.dll</code>
Windows Build	10.0.19045
SHA-256	84d34c90aa8322ea38a955069db302d007e9bf9f130bc64e197ceb0e3452335e
MD5	1417474a4615cb4736ad71e789eb42a2
PE Timestamp	0x358ECF00 (1998-06-22 21:39:12 UTC)

Table 5: Identification of the anomalous binary.

Observation. The binary’s import directory contains five descriptors. The fourth, resolving to `VERSION.dll`, exhibits a structural contradiction: its Original First Thunk (OFT) and First Thunk (FT) RVAs both point to well-formed, non-null locations, yet the descriptor’s resolved entry count is zero.

Field	Value
Descriptor Index	4 of 7
Imported Library	VERSION.dll
Descriptor File Offset	0xC1A3C
Original First Thunk RVA	0xCA2D4
Name RVA	0xCAC0E
First Thunk RVA	0xCA538
Resolved Entry Count	0

Table 6: Fourth `IMAGE_IMPORT_DESCRIPTOR` entry, showing valid thunk and name RVAs alongside a zero resolved entry count.

The remaining six descriptors resolve normally, with populated thunk entries and nonzero import counts, isolating this contradiction to a single descriptor rather than reflecting a corrupt import directory as a whole.

This binary required a targeted relaxation of the analysis pipeline’s thunk-walk logic to process past this descriptor without aborting, and is the only binary in the corpus exhibiting this particular combination of valid addressing metadata with an empty resolved thunk chain.

Structural Context.

Metric	Value
Disk File Size	839,680 bytes
In-Memory Size	876,544 bytes
Section Count	7 (1 executable, 3 writable)
File Alignment	0x200
Section Alignment	0x1000
Overlay Present	No

Table 7: Structural summary of `ir41_32original.dll`, for reference.

Discussion. Aside from the isolated import descriptor contradiction, the binary’s structure is otherwise unremarkable relative to the corpus. Its PE timestamp, `0x358ECF00` (1998-06-22), predates most binaries examined in this study by a wide margin and is consistent with the file’s identity as a legacy Indeo Video 4.1 codec component, retained in modern Windows builds for backward-compatibility with older multimedia content.

This offers a plausible, though not conclusively provable, explanation for how the contradiction could persist unnoticed, echoing the age-based rationale discussed above: the file likely predates linker conventions or validation tooling that would have caught an inconsistency of this kind at build time.

Whereas `ir41_32original.dll`’s anomaly is confined to a single descriptor within an otherwise well-formed import table, the final case in this section departs from that pattern entirely: rather than one contradictory field in an unremarkable binary, the following comparison involves a binary whose Windows 10 build is degenerate across nearly every structural dimension examined.

3.2.3 `onnxruntime.dll` — Degenerate Windows 10 Build vs. Well-Formed Windows 11 Build

Identification.

Field	Value
Windows 10 Build (10.0.19045)	
Filename	<code>onnxruntime.dll</code>
Path	<code>C:\Windows\System32\onnxruntime.dll</code>
SHA-256	<code>09ced31cad8547a9ee5dcf739565def2f4359075e56a7b699cc85971e0905864</code>
MD5	<code>a8283f0c13f681956daf369882e478eb</code>
PE Timestamp	Repro checksum <code>0x9E7F01A3</code> (no <code>TimeStamp</code>)
Disk Size	2,832 bytes

Continued on next page

Table 8 – continued from previous page

Field	Value
Windows 11 Build (10.0.26200)	
Filename	<code>onnxruntime.dll</code>
Path	<code>C:\Windows\System32\onnxruntime.dll</code>
SHA-256	<code>a9d3e0e13cadb011209013eee40ccbd6255d8a45090c7a12b635b7dde487934b</code>
MD5	<code>63ccb9b6b745f8c5eb3e1f6837595a5f</code>
PE Timestamp	Repro checksum 0x61CC70D5 (no TimeDateStamp)
Disk Size	10,572,960 bytes

Table 8: Identification of `onnxruntime.dll`, Windows 10 and Windows 11 builds.

Observation. The Windows 10 copy of this binary is degenerate across nearly every structural dimension examined, in contrast to the single-field contradictions seen in the two preceding case studies. Its disk size of 2,832 bytes is smaller than the Windows 11 build’s 10,572,960 bytes by a factor of approximately $3,733\times$ — the Windows 11 file is, by contrast, consistent with a genuine, fully built ONNX Runtime release.

Most critically, on the Windows 10 build both the Export Table and the Import Table data directory entries are zeroed (VA: `0x0`, Size: `0x0`), meaning the file exports no symbols and imports from no other module. Only five of fifteen data directories are populated on this build: Resource, Exception, Base Relocation, Debug, and Load Config.

The Windows 11 build, in contrast, populates eleven of fifteen directories, including a fully-formed Export Table (0x114 bytes, 276 entries’ worth of metadata) and Import Table (0x2A8 bytes) (Table 9).

Directory	Windows 10 (VA / Size)	Windows 11 (VA / Size)
Export Table	0x0 / 0x0	0x989C70 / 0x114
Import Table	0x0 / 0x0	0x989D84 / 0x2A8
Resource Table	0x6F0 / 0x3F8	0x9F9000 / 0x5E0
Exception Table	0x6B0 / 0x18	0x9AA000 / 0x4D970
Security Table	0x0 / 0x0	0x9FC600 / 0x18EA0
Base Relocation Table	0xAF0 / 0x18	0x9FA000 / 0xB950
Debug Directory	0x370 / 0x70	0x8FDE20 / 0x8C
TLS Table	0x0 / 0x0	0x8FE080 / 0x28
Load Config Table	0x3E0 / 0x108	0x7CA700 / 0x140
Import Address Table	0x0 / 0x0	0x7BC000 / 0x9D0
Delay Import Descriptor	0x0 / 0x0	0x9898D0 / 0xE0

Table 9: Data directory comparison (VA / Size), both builds of `onnxruntime.dll`. Architecture, Global Pointer, Bound Import, and CLR Runtime Header are zeroed in both builds and omitted here.

The Windows 10 build’s section table is similarly minimal: eight sections, virtual sizes as small as 8 bytes (`.CRT`), none exceeding 1,024 bytes, and a 66-byte `.text` section — its only executable section. The Windows 11 build’s seven sections are, by contrast, sized consistently with a production binary of its class, with `.text` alone spanning approximately 7.9 MB (Table 10).

Section	Windows 10 Virtual Size	Windows 11 Virtual Size
<code>.text</code>	66 bytes	8,102,939 bytes
<code>.rdata</code>	804 bytes	1,901,032 bytes
<code>.data</code>	24 bytes	114,888 bytes
<code>.pdata</code>	24 bytes	317,808 bytes
<code>.rsrc</code>	1,016 bytes	1,504 bytes
<code>.reloc</code>	24 bytes	46,416 bytes

Table 10: Section virtual size comparison, both builds. Windows 10 includes `.CRT` and `.00cfg` sections absent from the Windows 11 layout; Windows 11 includes a `.didat` section absent from Windows 10.

On the Windows 10 build, `FileAlignment` and `SectionAlignment` are both set to `0x10` (16 bytes), below the minimum value of `0x200` permitted by the PE/COFF specification.

The Windows 11 build uses the standard values (0x200 / 0x1000) and additionally sets `IMAGE_DLLCHARACTERISTICS_HIGH_ENTROPY_VA`, `DYNAMIC_BASE`, `NX_COMPAT`, and `GUARD_CF` — a normal, modern security-feature baseline entirely absent from the Windows 10 build’s available header data.

The Windows 10 build’s Base Relocation directory entry is internally consistent at the directory level: its `VA`: 0xAf0 correctly corresponds to the `.reloc` section’s location, and its declared size (0x18) matches that section’s size exactly. However, the relocation block contained within that section is itself malformed: the block’s leading Page RVA field — the first `DWORD` of every `IMAGE_BASE_RELOCATION` entry, which identifies the 4,096-byte page that the block’s subsequent entries apply to — is 0x00000000. This value falls within the region occupied by the DOS/NT headers rather than any valid code or data page; no properly linked binary references the header region as a relocation target. The anomaly is therefore not in the directory’s placement, which is correct, but in the content of the relocation block it points to.

Structural Context.

Metric	Windows 10	Windows 11
Disk File Size	2,832 bytes	10,572,960 bytes
Section Count	8 (1 exec, 1 write)	7 (1 exec, 2 write)
File Alignment	0x10	0x200
Section Alignment	0x10	0x1000
Export/Import Populated	No / No	Yes / Yes
Overlay Present	No	Yes (102,048 bytes)

Table 11: Structural comparison of `onnxruntime.dll` across builds.

Discussion. This case differs qualitatively from the preceding two. Rather than a single contradictory field embedded in an otherwise unremarkable binary, the Windows 10 copy is degenerate across nearly every structural dimension examined — absent import/export tables, sub-100-byte sections, a sub-specification alignment value, and a relocation block referencing an invalid page — while a properly formed, fully-featured build of the same-named file, some 3,700 times larger, coexists in the Windows 11 image with a normal modern security baseline. Both builds use reproducible-build checksums rather than classical `TimeStamp` values, indicating this is a project-wide build convention rather than a distinguishing detail between the two; the degenerate structure of the Windows 10 copy is therefore orthogonal to, not explained by, its build identification scheme.

Given the magnitude of the discrepancy — a 2,832-byte file where a multi-megabyte binary is expected, entirely absent of exports or imports — this is more consistent with a stub, placeholder, or incompletely staged build artifact left in the Windows 10 image than with corruption of a once-complete file. The exact mechanism by which such a file came to be installed at this path under the name `onnxruntime.dll` cannot be conclusively determined from static structure alone.

3.3 Case Study: `inpoutx64.sys`

`inpoutx64.sys` is a kernel-mode driver that exposes direct, low-level hardware access to unprivileged user-mode processes through a single `IRP_MJ_DEVICE_CONTROL` dispatch routine. Through a small set of IOCTL codes, the driver allows callers to read from and write to arbitrary I/O ports, as well as map arbitrary physical memory ranges into their virtual address space via `\Device\PhysicalMemory`. These capabilities are ordinarily reserved for kernel-mode code, yet the driver makes them accessible from ring 3 through a device object that imposes no access-control restrictions on which processes may open it. The remainder of this section traces how the driver was surfaced by the corpus methodology, what its provenance and trust chain reveal, and what its full IOCTL surface actually implements.

Discovery. `inpoutx64.sys` was surfaced by an automated corpus-wide check for RWX section permissions, a pattern rare enough in the sampled corpus to warrant flagging on its own: as reported in § 3.4 of [1], RWX section counts declined from 47 (0.10%) in Windows 10 to 19 (0.04%) in Windows 11, a reduction driven almost entirely by the elimination of a user-mode cluster (13 → 0), while RWX sections in kernel-mode drivers persisted at a reduced but still notable rate (34 → 19). The RWX-flagged driver set was cross-referenced against signer and driver category: of the drivers retaining an RWX-flagged INIT section in Windows 11, the overwhelming majority are storage-related (RAID, SATA, and chipset storage controllers) or Intel-signed profiling/sampling components, signed by known, established vendors — a

combination consistent with legitimate low-level buffer handling rather than an anomaly. Filtering that set down to binaries with an unrecognized signer left a single outlier: `inpoutx64.sys`, signed by “Red Fox UK Limited” and, unlike the rest of the RWX set, not a data-storage driver at all but an I/O port dispatcher. This combination — an uncommon section permission, an unfamiliar signer, and a functional category inconsistent with the rest of its permission cohort — prompted the deeper manual analysis that follows. Prior public analysis had independently identified this same driver as anomalous, reasoning from signer rarity alone and applying that reasoning manually to a single host [6], rather than as an automated, corpus-scale flag arising from section-permission statistics.

History. The driver’s PE header records a compile timestamp of 2008-10-17, consistent with Highresolution Enterprises’ own documented release of the first signed build [3]. `inpoutx64.sys` descends from Logix4u’s driver for 32-bit Windows (9x and NT-based systems through XP/2003), later ported to x64 by Highresolution Enterprises, which no longer actively maintains the project [3]. Signing was handled not by Highresolution itself but by a separate third party, Red Fox UK Limited; the author has stated publicly that as a hobbyist developer he lacked the means to obtain x64 driver-signing certificates himself, and that Red Fox UK signed the driver on his behalf as an accommodation [7].

Capability Delta Against Author-Provided Source. The original author was contacted directly and provided the source for an earlier, related project (`hwinterface`), predating the `inpoutx64.sys` rename. This source implements only a trivial `IRP_MJ_CREATE` handler and byte-granularity port I/O (`IOCTL_READ_PORT_UCHAR`, `IOCTL_WRITE_PORT_UCHAR`). The shipped, signed binary analyzed in this study implements substantially more: word- and dword-granularity port I/O, and a complete physical-memory mapping subsystem (`IOCTL_MAPPHYSTOLIN`, `IOCTL_UNMAPPHYSADDR`) built on `\Device\PhysicalMemory` and `HalTranslateBusAddress` — none of which appears in the author-provided source. The author states directly that he did not write this extended functionality. Because Authenticode validity requires the signed file to be byte-for-byte unmodified, this expansion must have occurred before signing, not as a later patch — placing its origin downstream of the disclosed author and upstream of, or concurrent with, the signing event. The binary/certificate evidence alone cannot establish *who* performed the expansion; this is inferred from the functional diff, the signature-validity constraint, and direct author testimony.

Signer Status. Red Fox UK Limited, the signing entity, was dissolved on 22 April 2014 per its Companies House record, over a decade before this analysis. This separates three properties of the binary’s trust chain that are normally conflated: the signing certificate’s own validity window (expired 2009), the signer’s continued legal existence (ended 2014), and the file’s Authenticode validity (still valid today via the countersigned timestamp discussed below). All three can diverge independently, and this binary illustrates exactly that.

Field	Value
Signer	CN=Red Fox UK Limited, OU=Development, OU=Digital ID Class 3 – Microsoft Software Validation v2, O=Red Fox UK Limited, L=London, S=London, C=GB
Issuer	CN=VeriSign Class 3 Code Signing 2004 CA
Not Before	9 October 2008
Not After	10 October 2009
Timestamp CA	VeriSign Time Stamping Services (valid 2007–2012)
Thumbprint	44A7C0E0680E59BD76F0077D60B7B3B55EF68518
Status	Valid (Authenticode, signature verified)

Table 12: Authenticode signature metadata for `inpoutx64.sys`.

Signature Analysis. The driver remains Authenticode-valid despite its signing certificate having expired over a decade ago. Table 12 shows a trusted, countersigned timestamp (VeriSign Time Stamping Services) accompanying the signature, which lets Windows verify the certificate against its validity window *at the time of signing* rather than at verification time — once timestamped, the signature remains valid indefinitely regardless of certificate expiry. This is a well-documented mechanism, and part of why Microsoft now maintains a separate vulnerable-driver blocklist (`HVCI/DriverSiPolicy.p7b`) rather than relying on cert expiry alone. A valid status also confirms the on-disk binary is byte-for-byte what was

signed, corroborating the capability-delta finding above: whatever was added beyond the author-provided source was added before this signing event.

Blocklist Correlation. Beyond code-signing validity, `inpoutx64.sys` is also subject to Microsoft’s separate vulnerable-driver blocklist, distributed via `DriverSiPolicy.p7b` and enforced under HVCI, Smart App Control, and S mode [8]. The current blocklist contains four deny rules referencing `inpoutx64.sys`, covering the file hash and page hash in both SHA-1 and SHA-256. These values did not match the file hash of the binary examined in this study when compared directly. Investigation of this discrepancy revealed the underlying mechanism: the blocklist’s hash rules are computed over the Authenticode hash — the value covered by the code-signing signature itself, which excludes the checksum field and certificate table — rather than the raw file hash. Cross-referencing against publicly catalogued `inpoutx64.sys` samples [4] identified three known builds by compile timestamp, shown in Table 13. The blocklist’s hash rules correspond to the Authenticode hash of a later, 2016-01-27-compiled build, not the 2008-10-17 build examined in this study, whose Authenticode hash differs entirely. Two known samples share the 2016-01-27 compile timestamp with distinct file hashes but an identical Authenticode hash, indicating the same underlying binary was signed more than once, likely to refresh an expiring or otherwise updated signature. This confirms that Microsoft’s blocklist coverage of `inpoutx64.sys` is build- and signature-specific rather than blanket: the driver examined here, despite sharing its name, vendor, and product version string with the blocklisted sample, is not itself denied by hash under the current policy. This distinction motivates the discussion of BYOVD risk below, where hash-based blocklisting is shown to be an incomplete mitigation against a driver family with multiple independently-signed builds in circulation.

Compile Timestamp	SHA-256 (file)	Authenticode SHA-256	Blocklisted
2008-10-17 (this study)	f8965fdc...ecf221b	d61ce587...d789a726	No
2016-01-27 (variant A)	2d83ccb1...62ebf44d	9f70169f...856852dd8	Yes
2016-01-27 (variant B)	f581decc...76acd1d29d53af	9f70169f...856852dd8	Yes

Table 13: Known `inpoutx64.sys` samples identified by compile timestamp, file hash, and Authenticode hash, cross-referenced against Microsoft’s current vulnerable-driver blocklist.

Technical details. Prior static analysis of `inpoutx64.sys` established that its `IRP_MJ_DEVICE_CONTROL` handler exposes eight IOCTL codes, five of which wrap the `IN/OUT` x86 instructions to permit arbitrary port I/O from user mode, and that the driver’s device object DACL grants access to `S-1-1-0 (Everyone)`, permitting any unprivileged process to issue these IOCTLs [6]. This accounts for six of the eight codes; the remaining two, `0x9C40201C` and `0x9C402020`, were left unexamined in that prior analysis. Independent reconstruction of the dispatch routine (Appendix A.2) confirms the control-flow structure reported previously and closes this gap. The finding described below is specific to the driver build examined in this study (SHA256 `f8965fdc...ecf221b`, compiled 2008-10-17); whether it is also present in the blocklisted 2016 builds discussed above in Table 13 was not independently verified.

The two unexamined codes turn out to implement physical-memory mapping and unmapping, not port I/O. The handler for `0x9C40201C` opens `\Device\PhysicalMemory` using a kernel-mode handle (`OBJ_KERNEL_HANDLE`) via `ZwOpenSection`, then maps the requested physical address range into the calling process’s address space via `ZwMapViewOfSection` with `PAGE_READWRITE | PAGE_NOCACHE` protection (Appendix A.3). The significance of the kernel-mode handle is that `\Device\PhysicalMemory` has been access-restricted from user mode since Windows XP SP2 precisely to prevent this kind of mapping; opening the section from kernel mode sidesteps that restriction entirely, and the DACL established above means the resulting capability is reachable by any unprivileged caller. No validation of the requested physical address range against installed memory or reserved regions was identified in the reconstructed code, so the capability is unconstrained as well as unauthenticated.

A further irregularity surfaces in how both codes handle their input. `0x9C40201C` and `0x9C402020` each pass the IOCTL’s `InputBufferLength` — a value fully controlled by the calling process — directly as the length argument to an internal, bounds-agnostic copy routine, with the destination being a fixed 32-byte region of the dispatch routine’s own stack frame (Appendix A.2). Nothing in either code path clamps this length against the size of that region before the copy executes. Static analysis of the routine’s raw disassembly (Appendix A.4) places the function’s saved return address 56 bytes beyond the base of this buffer, within a stack frame built via a direct `SUB RSP, 0x58` (no `push rbp` prologue) and containing no `/GS` stack-cookie check prior to return. Taken together, these properties are consistent with

a stack-based buffer overflow primitive reachable by an unprivileged caller. Dynamic verification and exploit development were not pursued, as doing so falls outside the scope of this paper’s corpus-level static methodology; this is noted as a limitation and a candidate for future targeted case-study work.

No assigned CVE for `inpoutx64.sys` specifically was identified in the course of this investigation. The SentinelOne disclosure frequently cited alongside it concerns a distinct driver — Dell’s `dbutil_2.3.sys` (CVE-2021-21551) — exhibiting the same general vulnerability class, unauthenticated port I/O and physical memory access, rather than the same binary [9]. As with the blocklist coverage discussed above, surface-level similarity between drivers in this family does not imply a shared identity, and claims tying a specific binary to a specific CVE should be verified against hash rather than name or vendor alone.

BYOVD Risk. The findings above illustrate why hash-based blocklisting alone is an incomplete defense against Bring Your Own Vulnerable Driver (BYOVD) attacks. `inpoutx64.sys` is denied by Microsoft’s blocklist only for two specific 2016-compiled builds; the 2008 build examined in this study — carrying an unauthenticated physical-memory mapping capability and, by the analysis above, a plausible stack-overflow primitive — is not covered, despite sharing the driver’s name, vendor, and product version string with the builds that are. An attacker or toolkit author need only source an unblocked build of a known-vulnerable driver family to sidestep denylist-based mitigations entirely, without requiring a new vulnerability at all. This is consistent with the broader BYOVD pattern documented across the driver ecosystem: hash-based rules are precise but brittle, and coverage tends to lag the full set of builds a vulnerable driver family has actually circulated in. A signature check alone would also have missed this driver entirely — the build examined here remains Authenticode-valid and is not denied by any currently published rule, yet exhibits the same capability class the blocklist exists to prevent.

Summary. Taken together, the discovery, signature, blocklist, and technical findings above form a single worked example of what the corpus-scale methodology in this paper is designed to surface: a driver flagged by an uncommon section permission combined with an unfamiliar signer, whose shipped binary contains substantial capability absent from the only source the original author could provide, whose signature remains valid a decade past its signer’s dissolution, whose blocklist coverage does not extend to the specific build in circulation here, and whose IOCTL surface, once manually reconstructed, exceeds what had previously been documented for it. No claim is made that this pattern is representative of the full corpus; it is presented as a single case study illustrating the kind of anomaly the methodology is built to detect, not as a generalized result.

The technical findings in this case study — particularly the stack-offset and frame-geometry analysis underlying the overflow primitive — rest on manual reconstruction of decompiled Windows kernel-mode code, a domain with substantial internal complexity (calling conventions, compiler-specific stack layout choices, and undocumented kernel structures among them). This analysis was conducted without the benefit of formal training in Windows internals or kernel driver development, and without dynamic verification to confirm the static findings against a running system. Reasonable effort was made to cross-check offsets, structure layouts, and API semantics against available documentation and prior published analysis where it existed, but domain-specific misinterpretation of a particular field, calling convention, or compiler behavior cannot be ruled out. Readers with deeper Windows kernel expertise are encouraged to treat the specific numeric claims in this section — the 56-byte offset in particular — as a starting point for independent verification rather than as a final, authoritative figure.

4 Limitation

The anomalies examined in this paper are illustrative rather than representative: four individual binaries and one unsigned-binary set, drawn from a corpus of 15,695, do not support claims about the prevalence of similar structural issues elsewhere in the Windows binary surface. Corpus-scale prevalence statistics are reported separately in [1].

The three detection routes described in § 2.2 differ in coverage as well as method. The catalog-dependent verification underlying the unsigned binaries (§ 3.1) carries its own caveats, noted there. The field-level anomalies in § 3.2 were surfaced incidentally, as a byproduct of parser hardening during corpus construction, rather than through a criterion applied uniformly across the corpus; other binaries with similarly unconventional fields may exist but simply did not happen to trigger a parser edge case. The `inpoutx64.sys` detection route, by contrast, was a targeted corpus-wide scan, but one specific to a single signal (RWX permissions cross-referenced with signer and category) and would not surface anomalies

expressed through other structural properties. Across all three routes, binaries not captured may exist in the corpus and are not claimed to be exhaustively enumerated by this study.

The `inpoutx64.sys` reconstruction rests on manual decompilation and disassembly of the shipped binary itself via Ghidra, not on recompiling or matching against any existing source release, and does not establish bit-for-bit equivalence with the original build’s toolchain. Inferred compiler flags and build settings are therefore best-effort reconstructions based on observable disassembly and PE characteristics, not confirmed against the original build environment.

For limitations concerning corpus scope, collection methodology, and the extraction pipeline, see § 4 of [1].

5 Related Work

BYOVD and vulnerable driver detection at scale. Monzani et al. [10] characterize how malware operators source and abuse kernel drivers under the Bring Your Own Vulnerable Driver pattern, and Gupta et al.’s POPKORN [11] applies automated analysis to surface vulnerable Windows kernel drivers at scale. Both share this paper’s concern with the driver attack surface, but target discovery of previously unknown vulnerable drivers across a population; this paper instead takes a single already-known driver family and shows that build-level variance undermines hash-based mitigation even after the family is known and blocklisted, illustrated concretely by SentinelOne’s independent disclosure of the same vulnerability class in a different driver, `dbutil.2.3.sys` [9]. The two perspectives are complementary: discovery-at-scale work determines which drivers should be blocked; the case study here shows that blocking is only as complete as the set of builds a blocklist actually enumerates.

Driver trust and signing infrastructure. This paper’s signature and blocklist analysis builds directly on documented Authenticode mechanics [5], Microsoft’s current certificate trust model [12], and the published vulnerable-driver blocklist enforced via HVCI and `DriverSiPolicy.p7b` [8]. Where that documentation describes the mechanism, § 3.3 applies it to a concrete case in which certificate expiry, signer dissolution, and Authenticode validity diverge from one another – a divergence the documentation establishes as possible but does not itself demonstrate on a real, currently-circulating binary.

Prior analysis of `inpoutx64.sys`. `inpoutx64.sys` has been examined before: Starke [6] manually analyzed a single host’s copy of the driver and flagged it as suspicious based on signer rarity, and the driver’s author has documented its provenance and signing history directly [3, 7]. This paper’s case study differs from Starke’s in how the driver was found – an automated, corpus-scale RWX-permission scan rather than manual suspicion applied to one host (§ 3.3, *Discovery*) – and extends the prior technical analysis by resolving two IOCTL codes left unexamined previously and by identifying a build-specific gap in current blocklist coverage, cross-referenced against the LOLDrivers catalogue [4].

Corpus-scale Windows driver analysis. Broader precedent for automated, corpus-scale analysis of the Windows driver surface – including IOCTL dispatch-surface recovery [13] and target selection for vulnerability research [14] – is discussed further in [1], whose corpus and pipeline this paper shares (§ 2.1). This paper applies the same corpus-scale orientation specifically to structural-anomaly detection rather than IOCTL surface recovery or target selection, and § 3.3 demonstrates one such anomaly carried through to a full case study.

6 Conclusion and Future Work

None of the anomalies examined in § 3 are explained by a single property of the binaries themselves. Signing status, structural convention, and detection coverage are treated in this paper as claims about a binary’s present trustworthiness, but each of the cases examined here shows that claim decoupling from the binary’s actual behavior over time. A signature establishes that a file has not changed since it was signed, not that it continues to be reviewed, nor that what it does was ever fully accounted for at the time. `inpoutx64.sys` remains Authenticode-valid today despite its signing certificate having expired over a decade ago and its signer having been dissolved since 2014, because a timestamp countersignature freezes its trust evaluation to a single moment in 2008 rather than reflecting anything about the driver’s present state.

This drift is not confined to third-party components, nor is it evidence of carelessness so much as a consequence of what maintaining a closed catalog of tens of thousands of binaries actually requires. `MaxxAudioEQ64.dll` and `ir41_32original.dll` carry PE timestamps over a decade and, in the latter case, over two decades old, and persist in the corpus essentially unmodified precisely because nothing has forced either to be rebuilt against more recent toolchain conventions. No single review pass scales to every component in a catalog composed of pieces authored years apart, some retained only for backward compatibility; the corpus-scale methodology used throughout this paper exists because binary-by-binary review does not scale to an ecosystem of this size.

The cross-build comparison in this corpus shows real improvement alongside this persistence, not a contradiction of it. Of the fourteen binaries examined in § 3.1, nine are absent entirely from Windows 11, while the remaining five persist as unsigned across both builds – improvement at the level of the majority, not a clean resolution. `onnxruntime.dll`’s degenerate Windows 10 build, by contrast, gives way to a fully-formed, modern build in Windows 11 – consistent with the broader hardening trend documented at corpus scale in [1]. Windows 11 benefits from roughly a decade of accumulated build-process maturity beyond Windows 10, and that maturity is visible in these findings. It does not extend to every component, however: legacy binaries retained for backward compatibility are, by definition, not subject to whatever process improved the rest of the catalog, and a decade of build-up applies to the components that get rebuilt, not to the ones deliberately left alone. The trend is real at the level of the corpus; it is not a guarantee that every legacy component riding along with it has been swept up in the same improvement – as the five persistently unsigned binaries in § 3.1 themselves demonstrate.

`inpoutx64.sys` illustrates what this gap looks like outside of static analysis alone. Several third-party AV/EDR products flag the driver; Windows Defender, running natively on the platform the driver targets, does not. Microsoft’s own vulnerable-driver blocklist covers two 2016-compiled builds by Authenticode hash but not the 2008-compiled build examined in this study, despite that build sharing the driver’s name, vendor, and product version string with the builds that are covered. None of the trust signals ordinarily available to an end system – signature validity, blocklist membership, native antimalware detection – surfaced this driver; only the corpus-scale structural methodology applied here did.

The stack-based buffer overflow primitive identified in § 3.3 was established through static reconstruction alone; dynamic verification against a running system, and formal disclosure should that verification hold, remain open work. More broadly, the gap documented here for a single driver family motivates a wider question this paper’s methodology was not built to answer at scale: how many other binaries in the corpus carry a similarly stale relationship between their actual capability and the trust signals a system administrator would ordinarily rely on. Extending the detection routes in § 2 to correlate structural anomalies directly against blocklist and antimalware coverage, rather than treating `inpoutx64.sys` as an individually investigated case, is a natural continuation of this work.

References

- [1] Abdelrahman Khalid. *A Corpus-Scale Static Analysis of Windows 10/11 System Binaries: Characterization and Cross-Build Differences*. Manuscript in preparation; expected release October 2026. URL is a placeholder pending publication. 2026. URL: <https://github.com/OxNull111/PLACEHOLDER> (visited on 08/09/2026).
- [2] Abdelrahman Khalid. *win10-11-binary-corpus-research*. Research repository for this study (analysis scripts and supporting tooling). 2026. URL: <https://github.com/OxNull111/win10-11-binary-corpus-research> (visited on 08/05/2026).
- [3] Phillip Gibbons. *InpOut32 and InpOutx64*. Highresolution Enterprises. 2026. URL: <https://www.highrez.co.uk/downloads/inpout32/default.htm> (visited on 08/05/2026).
- [4] Michael Haag. *inpoutx64.sys*. Entry created 2023-01-09, last updated 2026-07-10. LOLDrivers. 2023. URL: <https://www.loldrivers.io/drivers/91ff1575-9ff2-46fd-8bfe-0bb3e3457b7f/> (visited on 08/05/2026).
- [5] Microsoft. *Authenticode Digital Signatures – Windows Drivers*. Microsoft Learn. 2025. URL: <https://learn.microsoft.com/en-us/windows-hardware/drivers/install/authenticode> (visited on 07/28/2026).
- [6] Nicholas Starke. *Inpoutx64.sys Windows Driver Analysis*. StarkeBlog. May 2021. URL: <https://starkeblog.com/windows/kernel/driver/2021/05/15/inpoutx64.sys-windows-driver-analysis.html> (visited on 08/06/2026).

- [7] Phillip Gibbons. *Forum Post on InpOutx64 Signing*. Highresolution Enterprises Forums. 2026. URL: <https://forums.highrez.co.uk/> (visited on 08/05/2026).
- [8] Microsoft. *Microsoft Recommended Driver Block Rules*. Accessed via App Control for Business documentation; page updated 2026-05-04. Microsoft Learn. 2026. URL: <https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/design/microsoft-recommended-driver-block-rules> (visited on 08/05/2026).
- [9] SentinelOne Labs. *CVE-2021-21551: Hundreds of Millions of Dell Computers at Risk Due to Multiple BIOS Driver Privilege Escalation Flaws*. SentinelOne. 2021. URL: <https://www.sentinelone.com/labs/cve-2021-21551-hundreds-of-millions-of-dell-computers-at-risk-due-to-multiple-bios-driver-privilege-escalation-flaws/> (visited on 08/06/2026).
- [10] Andrea Monzani et al. “Unveiling BYOVD Threats: Malware’s Use and Abuse of Kernel Drivers”. In: *33rd Network and Distributed System Security Symposium (NDSS)*. Feb. 2026. URL: <https://www.ndss-symposium.org/wp-content/uploads/2026-s1491-paper.pdf> (visited on 08/09/2026).
- [11] Rajat Gupta et al. “POPKORN: Popping Windows Kernel Drivers at Scale”. In: *Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC)*. ACM, Dec. 2022. URL: <https://doi.org/10.1145/3564625.3564631> (visited on 08/09/2026).
- [12] Microsoft. *Artifact Signing Trust Models*. Service formerly known as Azure Code Signing, then Trusted Signing. Microsoft Learn. 2026. URL: <https://learn.microsoft.com/en-us/azure/artifact-signing/concept-trust-models> (visited on 08/08/2026).
- [13] Michael J. Bommarito II. *The Windows IOCTL Census: A Corpus-Scale, Multi-Architecture Database of the Driver Control-Code Surface*. June 2026. arXiv: [2606.07732](https://arxiv.org/abs/2606.07732) [cs.SE].
- [14] Michael J. Bommarito II. *Needles at Scale: LLM-Assisted Target Selection for Windows Vulnerability Research*. May 2026. arXiv: [2606.01364](https://arxiv.org/abs/2606.01364) [cs.SE].

A Decompiled Listings

A.1 Driver Entry

```

1 NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject) {
2     NTSTATUS status;
3     UNICODE_STRING deviceName, symbolicLinkName;
4     PDEVICE_OBJECT deviceObject;
5     WCHAR deviceNameBuf[20]; // L"\\Device\\inpoutx64"
6     WCHAR symbolicLinkBuf[24]; // L"\\DosDevices\\inpoutx64"
7
8     wcsncpy(deviceNameBuf, L"\\Device\\inpoutx64", 0x12);
9     wcsncpy(symbolicLinkBuf, L"\\DosDevices\\inpoutx64", 0x16);
10
11     RtlInitUnicodeString(&deviceName, deviceNameBuf);
12     RtlInitUnicodeString(&symbolicLinkName, symbolicLinkBuf);
13
14     status = IoCreateDevice(DriverObject, 0, &deviceName,
15                           FILE_DEVICE_UNKNOWN, 0, FALSE,
16                           &deviceObject);
17
18     if (NT_SUCCESS(status)) {
19         status = IoCreateSymbolicLink(&symbolicLinkName, &deviceName);
20         if (NT_SUCCESS(status)) {
21             DriverObject->MajorFunction[IRP_MJ_CREATE] = DispatchCreate;
22             DriverObject->MajorFunction[IRP_MJ_DEVICE_CONTROL] =
23                 DispatchDeviceControl;
24             DriverObject->DriverUnload = DriverUnload;
25             status = STATUS_SUCCESS;
26         }
27     }
28     return status;
29 }

```

Listing 1: Reconstructed `DriverEntry`, invoked from the compiler-generated `entry` thunk after standard /GS stack-cookie initialization (omitted)

A.2 Device Control Dispatcher

```
1 NTSTATUS DispatchDeviceControl(PDEVICE_OBJECT DeviceObject, PIRP Irp) {
2     PIO_STACK_LOCATION IrpSp = IoGetCurrentIrpStackLocation(Irp);
3     ULONG OutputBufferLength = IrpSp->Parameters.DeviceIoControl.OutputBufferLength;
4     ULONG InputBufferLength = IrpSp->Parameters.DeviceIoControl.InputBufferLength;
5     PUCCHAR Buffer = (PUCCHAR)Irp->AssociatedIrp.SystemBuffer;
6     NTSTATUS status = STATUS_SUCCESS;
7     ULONG_PTR information = 0;
8
9     switch (IrpSp->Parameters.DeviceIoControl.IoControlCode) {
10         case IOCTL_READ_PORT_UCHAR: // 0x9C402004
11             if (OutputBufferLength > 1 && InputBufferLength != 0)
12                 Buffer[0] = __inbyte(*(USHORT *)Buffer);
13             information = 1;
14             break;
15
16         case IOCTL_WRITE_PORT_UCHAR: // 0x9C402008
17             if (InputBufferLength > 2) {
18                 __outbyte(*(USHORT *)Buffer, *(UCHAR *) (Buffer + 2));
19                 information = 10;
20             }
21             break;
22
23         case IOCTL_READ_PORT_USHORT: // 0x9C40200C
24             if (OutputBufferLength > 1 && InputBufferLength > 1)
25                 *(USHORT *)Buffer = __inword(*(USHORT *)Buffer);
26             information = 2;
27             break;
28
29         case IOCTL_WRITE_PORT_USHORT: // 0x9C402010
30             if (InputBufferLength > 3) {
31                 __outword(*(USHORT *)Buffer, *(USHORT *) (Buffer + 2));
32                 information = 10;
33             }
34             break;
35
36         case IOCTL_READ_PORT_ULONG: // 0x9C402014
37             if (OutputBufferLength > 3 && InputBufferLength > 3)
38                 *(ULONG *)Buffer = __indword(*(USHORT *)Buffer);
39             information = 4;
40             break;
41
42         case IOCTL_WRITE_PORT_ULONG: // 0x9C402018
43             if (InputBufferLength > 7)
44                 __outdword(*(USHORT *)Buffer, *(ULONG *) (Buffer + 4));
45             break;
46
47         // InputBufferLength used directly as copy length; no bound
48         // against the destination stack buffer.
49         case IOCTL_MAPPHYSTOLIN: { // 0x9C40201C
50             if (InputBufferLength == 0) {
51                 status = STATUS_INVALID_PARAMETER;
52                 break;
53             }
54             PHYS_MEM_STRUCT req; // { PhysAddr, Size }
55             RtlCopyMemory(&req, Buffer, InputBufferLength);
56
57             PVOID mappedVA;
58             HANDLE sectionHandle;
59             status = MapPhysicalMemoryRange(req.PhysAddress, req.Size,
60                                             &mappedVA, &sectionHandle);
61             if (status >= 0) {
62                 req.MappedVA = mappedVA;
63                 req.SectionHandle = sectionHandle;
64                 RtlCopyMemory(Buffer, &req, InputBufferLength);
65                 information = InputBufferLength;
66             }
67             break;
68         }
69
70         // Same unchecked InputBufferLength copy as above.
```

```

71     case IOCTL_UNMAPPHYSADDR: { // 0x9C402020
72         if (InputBufferLength == 0) {
73             status = STATUS_INVALID_PARAMETER;
74             break;
75         }
76         PHYS_MEM_STRUCT req;
77         RtlCopyMemory(&req, Buffer, InputBufferLength);
78
79         status = ZwUnmapViewOfSection((HANDLE)-1, req.MappedVA);
80         ZwClose(req.SectionHandle);
81         break;
82     }
83
84     default:
85         status = STATUS_NOT_IMPLEMENTED;
86 }
87
88 Irp->IoStatus.Information = information;
89 Irp->IoStatus.Status = status;
90 IoCompleteRequest(Irp, IO_NO_INCREMENT);
91 return status;
92 }

```

Listing 2: Reconstructed DispatchDeviceControl. The IOCTL_MAPPHYSTOLIN/IOCTL_UNMAPPHYSADDR cases pass InputBufferLength directly as a copy length with no bound against the destination stack buffer; see § 3.3, *Technical details*.

A.3 Physical Memory Mapping Routine

```

1  NTSTATUS MapPhysicalMemoryRange(
2      PHYSICAL_ADDRESS PhysAddrBase,
3      SIZE_T Length,
4      PVOID *MappedVA,
5      HANDLE *SectionHandle) {
6
7      UNICODE_STRING sectionName;
8      OBJECT_ATTRIBUTES objAttr;
9      PVOID objectPtr = NULL;
10     PVOID viewBase = NULL;
11     SIZE_T viewSize;
12     PHYSICAL_ADDRESS endAddr;
13     NTSTATUS status;
14
15     // Unresolved by the decompiler; type/origin not independently
16     // confirmed. Retained here only to keep the listing compilable
17     // in form -- not verified against the original binary.
18     ULONG addrSpace;
19     PHYSICAL_ADDRESS someAlignedBase;
20
21     RtlInitUnicodeString(&sectionName, L"\\Device\\PhysicalMemory");
22     InitializeObjectAttributes(&objAttr, &sectionName,
23                               OBJ_CASE_INSENSITIVE | OBJ_KERNEL_HANDLE,
24                               NULL, NULL);
25
26     *SectionHandle = 0;
27     status = ZwOpenSection(SectionHandle, SECTION_ALL_ACCESS, &objAttr);
28     if (status < 0) goto fail;
29
30     status = ObReferenceObjectByHandle(*SectionHandle, SECTION_ALL_ACCESS,
31                                       NULL, KernelMode, &objectPtr, NULL);
32     if (status < 0) goto fail;
33
34     endAddr.QuadPart = PhysAddrBase.QuadPart + Length;
35
36     // No validation of PhysAddrBase/Length against installed memory
37     // or reserved regions prior to this translation/mapping.
38     if (!HalTranslateBusAddress(Isa, 0, PhysAddrBase, &addrSpace, &PhysAddrBase) ||
39         !HalTranslateBusAddress(Isa, 0, endAddr, &addrSpace, &endAddr))
40         goto fail;
41

```

```

42     viewSize = endAddr.QuadPart - PhysAddrBase.QuadPart;
43     if (viewSize == 0) goto fail;
44
45     status = ZwMapViewOfSection(
46         *SectionHandle, (HANDLE)-1, &viewBase,
47         0, viewSize, &PhysAddrBase, &viewSize,
48         ViewShare, 0, PAGE_READWRITE | PAGE_NO_CACHE);
49
50     if (status >= 0) {
51         *MappedVA = (PVOID)((ULONG_PTR)viewBase +
52             ((ULONG)PhysAddrBase.LowPart - (ULONG)someAlignedBase.LowPart));
53         return status;
54     }
55
56 fail:
57     ZwClose(*SectionHandle);
58     return status;
59 }

```

Listing 3: Reconstructed MapPhysicalMemoryRange. `addrSpace` and `someAlignedBase` are declared as unresolved decompilation artifacts (see inline note) rather than variables confirmed present in the original binary

A.4 Ghidra Disassembly: Stack Frame Geometry

```

1  ; -- prologue --
2  000112f0 48 83 ec 58 SUB RSP,0x58
3
4  ; [...]
5
6  ; -- case 0x9c40201c (IOCTL_MAPPHYSTOLIN): copy destination setup --
7  0001140d 48 89 6c 24 78 MOV qword ptr [RSP+0x78],RBP
8  00011412 48 8b ea MOV RBP,param_2
9  00011415 48 8d 4c 24 20 LEA param_1 => local_38,[RSP+0x20]
10 0001141a 48 8b d3 MOV param_2,RBX
11 0001141d e8 8e 02 00 00 CALL FUN_000116b0
12
13 ; [...]
14
15 ; -- epilogue --
16 000114cb 48 83 c4 58 ADD RSP,0x58
17 000114cf c3 RET

```

Listing 4: Excerpt of DispatchDeviceControl (FUN_000112f0) disassembly as rendered by Ghidra, substantiating the stack-offset calculation in § 3.3, *Technical details*. Labels of the form `param_N` and `local_N` are Ghidra’s inferred parameter and stack-variable names, not literal register operands; `param_1 => local_38` denotes Ghidra’s annotation that `param_1` is tracked through the stack slot `local_38`. The prologue establishes a fixed 0x58-byte frame with no `push rbp`; the `IOCTL_MAPPHYSTOLIN` (0x9c40201c) handler establishes its copy destination at `[RSP+0x20]`; the epilogue confirms the same 0x58-byte frame is torn down immediately before `RET`, placing the saved return address 56 bytes above the copy destination